

Relazione Homework 2 Sistemi Distribuiti e Big Data OpenSky Flight Monitor

Giovanni Maria Contarino 1000007029

Alessia Provvidenza Tomarchio 1000005160

Sommario

Introduzione ed Evoluzione degli obiettivi.....	3
Aggiornamento Architettuale: Networking e Sicurezza.....	5
Isolamento di rete tramite i Docker Networks	5
API Gateway (Nginx)	6
Nuovi Componenti.....	7
Introduzione di un Message Broker: Kafka.....	7
Invio notifiche reali tramite server SMTP	8
Circuit Breaker	8
Evoluzione della Logica di Business	9
Gestione delle Soglie.....	9
Consistenza e Pulizia.....	12

Introduzione ed Evoluzione degli obiettivi

Il seguente documento illustra l'evoluzione e lo sviluppo architetturale del sistema da noi denominato "OpenSky Flight Monitor", descritto nella precedente relazione. Nella prima versione ci siamo concentrati nella creazione dei servizi (Core Services) come lo User Manager e il Data Collector, questa seconda iterazione ha l'obiettivo di risolvere alcune criticità strutturali già citate nella precedente relazione.

Le principali innovazioni introdotte riguardano:

1. **Isolamento di Rete:** Introduzione dei Docker Networks per proteggere i microservizi dall'esterno.
2. **API Gateway:** Centralizzazione degli accessi tramite Nginx per eliminare l'esposizione diretta delle porte dei servizi.
3. **Message Broker:** Introduzione di Apache Kafka per disaccoppiare la logica di raccolta dati da quella di allertamento.
4. **Resilienza:** Implementazione del pattern Circuit Breaker per gestire i guasti delle API esterne.
5. **Notifiche Reali:** Sistema di invio email tramite protocollo SMTP autenticato.

Di seguito viene riportato il diagramma che illustra i componenti del sistema e le tecnologie adottate, e successivamente, il Diagramma di Sequenza (figura 2), che mostra il flusso temporale delle nuove iterazioni introdotte con questa revisione, tra i componenti interni alla rete Docker (User Manager, Data Collector, Database, Notifier System e Alert System) e il mondo esterno (Client, OpenSky Network API e Gmail SMTP).

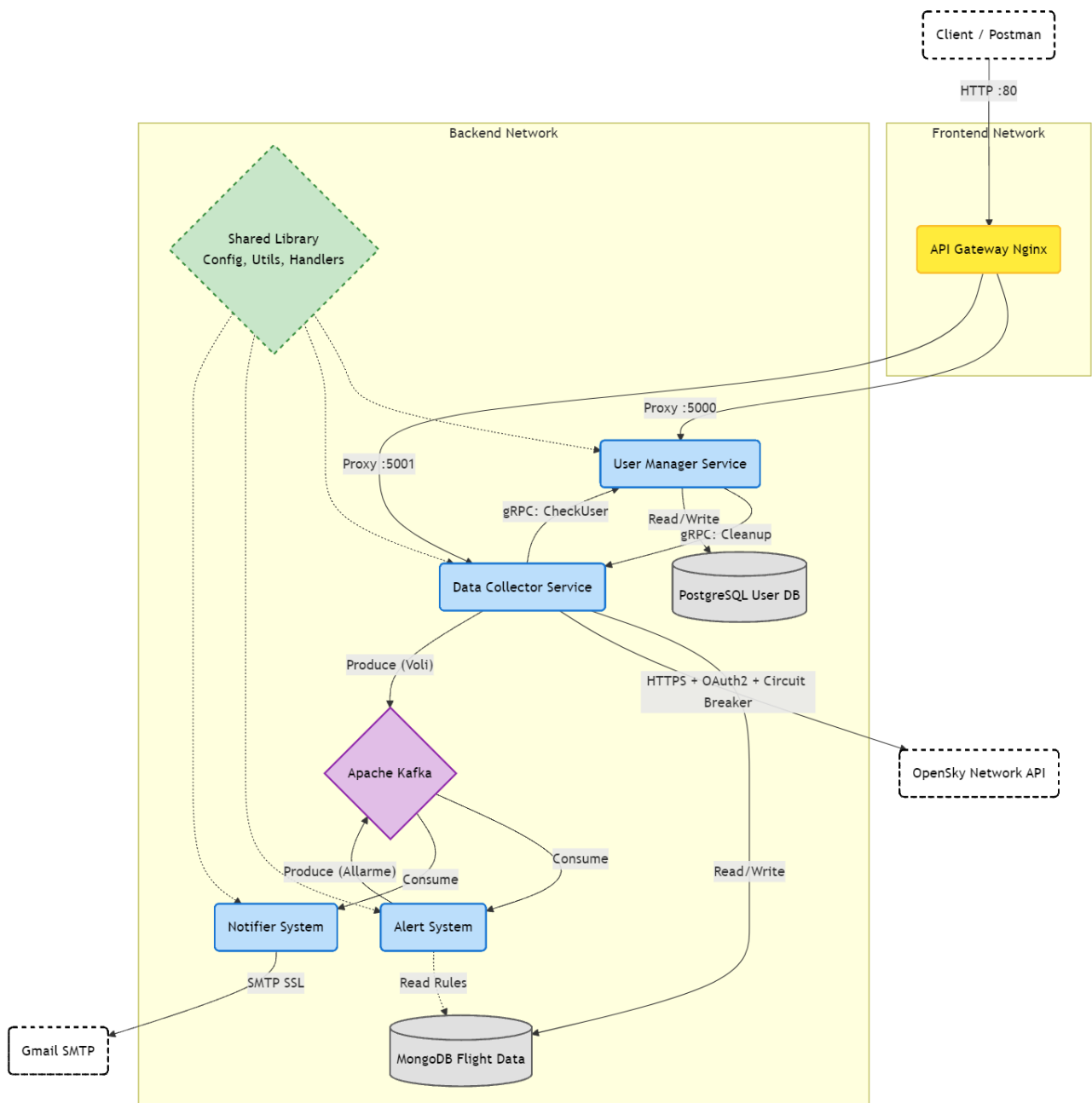


Figura 1. Diagramma architetturale che mostra tutti i micro-servizi coinvolti all'interno del nostro sistema aggiornato al secondo homework.

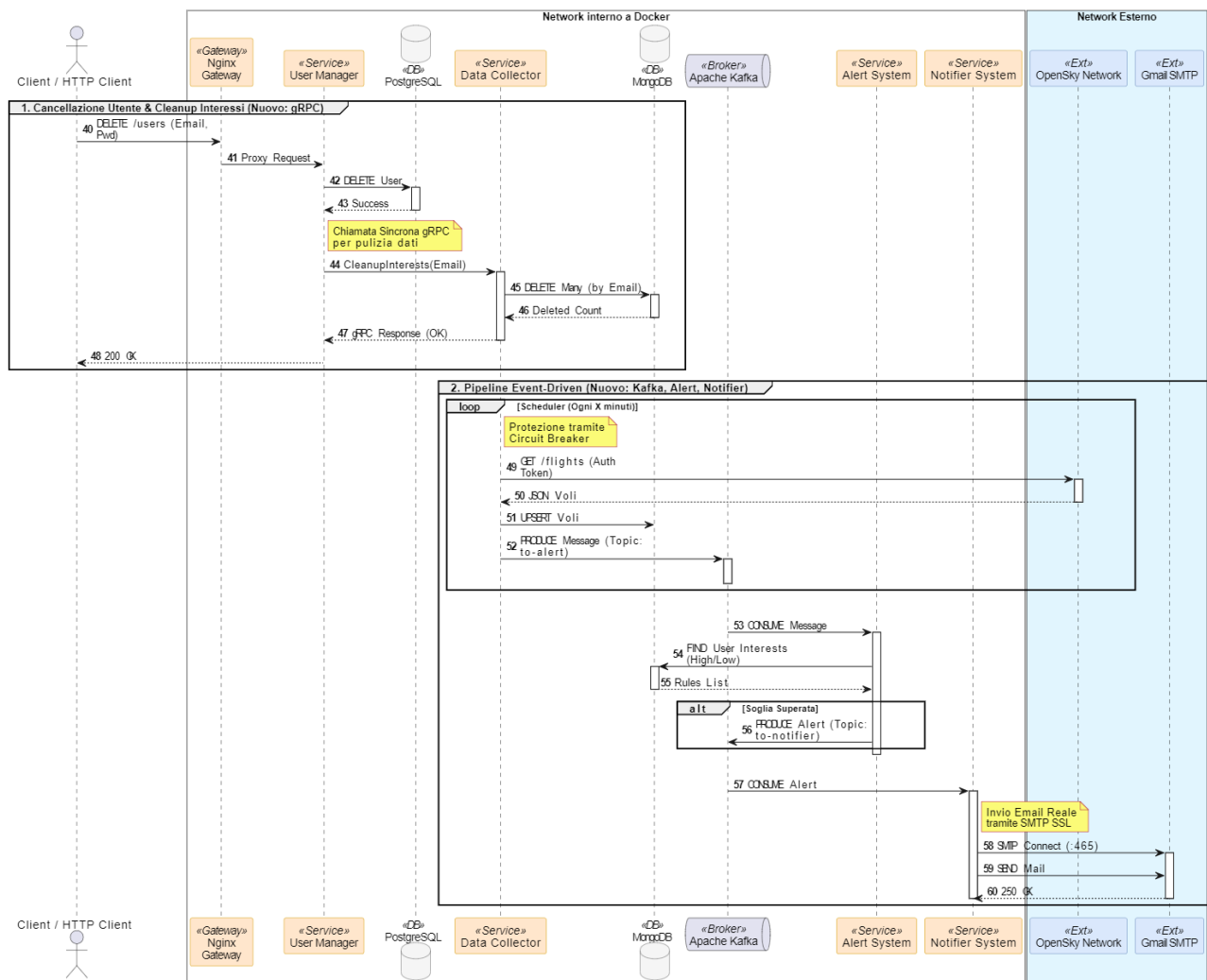


Figura 2. Diagramma che mostra nel dettaglio le interazioni, sia tra componenti dell'applicazione che tra l'applicazione il mondo esterno.

Aggiornamento Architettuale: Networking e Sicurezza

Isolamento di rete tramite i Docker Networks

Nella prima versione del progetto, tutti i container operavano sulla rete di default o espongono le proprie porte direttamente all'host. Questa configurazione presentava un rischio di sicurezza, in quanto i database e i servizi backend erano potenzialmente raggiungibili dall'esterno.

Nella versione attuale, abbiamo ristrutturato la topologia di rete definendo due network distinti nel docker-compose:

- **Backend Network:** È una rete privata e isolata dall'esterno. Qui risiedono i microservizi (User Manager, Data Collector), i database (PostgreSQL, MongoDB), il Broker Kafka e i nuovi servizi di Alert e Notifier. I componenti in questa rete possono comunicare tra loro velocemente, ma non sono accessibili direttamente dall'host o da client esterni.
- **Frontend Network:** È la rete pubblica di ingresso. Attualmente ospita l'**API Gateway (Nginx)**, l'unico componente autorizzato a ricevere traffico dall'esterno (porta 80).

Questa separazione garantisce che un attaccante esterno non possa interagire direttamente con i database o i microservizi, ma debba obbligatoriamente passare attraverso il filtro dell'API Gateway.

API Gateway (Nginx)

Come ipotizzato nella sezione "Variazioni dello Schema Architetture" del precedente documento, abbiamo introdotto **Nginx** come Reverse Proxy. Il gateway ascolta sulla porta 80 e smista le richieste ai servizi corretti nella rete di backend:

- Le chiamate /users vengono ruotate verso lo User Manager.
- Le chiamate /flights, /statistics e /interests verso il Data Collector.

Questa architettura centralizza la gestione degli accessi e semplifica il client, che ora dialoga con un unico endpoint (localhost:80) invece di dover conoscere le porte specifiche di ogni servizio (5000, 5001).

Nuovi Componenti

Introduzione di un Message Broker: Kafka

Per superare i limiti della comunicazione sincrona, abbiamo introdotto un'architettura a eventi basata su **Apache Kafka**. Il flusso di monitoraggio è stato suddiviso in tre fasi distinte e disaccoppiate:

1. **Data Collector (Producer):** Una volta scaricati i voli, pubblica i dati grezzi sul topic `to-alert-system`.
2. **Alert System (Consumer/Producer):** Un nuovo microservizio che legge i dati di volo, recupera le regole degli utenti da MongoDB (soglie numero voli) e verifica se le condizioni sono soddisfatte. In caso positivo, pubblica un evento sul topic `to-notifier`.

```
Sono stati ricevuti dall'aeroporto EGLL in (departure): 78 voli.  
Allarme per giovanni.contarino.gc@gmail.com: SOGLIA ALTA SUPERATA: 78 voli > 10.0  
Inviati 1 allarmi al topic to-notifier da parte del Producer Kafka
```

Figura 3. Log di funzionamento dell'Alert System.

3. **Notifier System (Consumer):** Ascolta gli eventi di allarme e si occupa esclusivamente dell'invio della notifica finale.

```
Notifier avviato: in ascolto su: to-notifier  
-----  
[EMAIL SENT]  
TO:      giovanni.contarino.gc@gmail.com  
SUBJECT: Alert for Airport EGLL  
BODY:    SOGLIA ALTA SUPERATA: 78 voli > 10.0  
          (Valore rilevato: 78)  
-----  
Email inviata correttamente al server SMTP.
```

Figura 4. Log di funzionamento del Notifier.

Questa catena garantisce che il Data Collector non venga rallentato dalle operazioni di invio email e che il sistema sia resiliente a picchi di traffico.

Invio notifiche reali tramite server SMTP

Il microservizio **Notifier** è stato implementato per inviare email reali. Utilizza la libreria `smtplib` di Python e si interfaccia con il server SMTP di Gmail (porta 465 SSL). Per garantire la sicurezza, l'autenticazione avviene tramite **Password per le App** generate specificamente per il progetto tramite l'Account Google, evitando l'uso della password personale dell'utente.



Figura 5. Esempio di email ricevuta al seguito di un superamento delle soglie.

Circuit Breaker

Considerando la dipendenza critica dalle API esterne di OpenSky, abbiamo implementato il pattern **Circuit Breaker** all'interno del Data Collector. Il componente monitora le chiamate verso OpenSky:

- **Stato Closed:** Funzionamento normale.

- **Stato Open:** Dopo 3 fallimenti consecutivi (es. errore 429, Too Many Requests), il circuito si "apre" impedendo ulteriori chiamate per 60 secondi. Questo evita di sovraccaricare un servizio esterno già in sofferenza e velocizza la risposta del nostro sistema.
- **Stato Half-Open:** Dopo il timeout, il sistema tenta una singola chiamata di prova per verificare se il servizio esterno è tornato disponibile.

```
Press CTRL+C to quit
Fetch dei voli in corso (Arrivi e Partenze)
Errore rilevato (1/3): HTTPSConnectionPool(host='fake_endpoint', port=443): Max retries exceeded with url: /fake/fake/fake?airport=EGLL&begin=1766136266&end=1766165066 (Caused by NameResolutionError("HTTPSConnection(host='fake_endpoint', port=443): Failed to resolve 'fake_endpoint' ([Errno -5] No address associated with hostname)"))
Err EGLL: HTTPSConnectionPool(host='fake_endpoint', port=443): Max retries exceeded with url: /fake/fake/fake?airport=EGLL&begin=1766136266&end=1766165066 (Caused by NameResolutionError("HTTPSConnection(host='fake_endpoint', port=443): Failed to resolve 'fake_endpoint' ([Errno -5] No address associated with hostname)"))
```

Figura 6. Log di funzionamento del Circuit Breaker.

```
Err EGLL: HTTPSConnectionPool(host='endpoint_falso', port=443): Max retries exceeded with url: /falso/falso/falso?airport=EGLL&begin=1766136266&end=1766165066 (Caused by NameResolutionError("HTTPSConnection(host='endpoint_falso', port=443): Failed to resolve 'endpoint_falso' ([Errno -2] Name or service not known)"))
Errore rilevato (3/3): HTTPSConnectionPool(host='fake_endpoint', port=443): Max retries exceeded with url: /fake/fake/fake?airport=KJFK&begin=1766136266&end=1766165066 (Caused by NameResolutionError("HTTPSConnection(host='fake_endpoint', port=443): Failed to resolve 'fake_endpoint' ([Errno -2] Name or service not known)"))
Circuit Breaker: SOGLIA RAGGIUNTA -> CIRCUITO APERTO!
Err KJFK: HTTPSConnectionPool(host='fake_endpoint', port=443): Max retries exceeded with url: /fake/fake/fake?airport=KJFK&begin=1766136266&end=1766165066 (Caused by NameResolutionError("HTTPSConnection(host='fake_endpoint', port=443): Failed to resolve 'fake_endpoint' ([Errno -2] Name or service not known)"))
CIRCUIT BREAKER APERTO: Chiamata bloccata per KJFK. Circuito APERTO. Riprova tra 60.0s
```

Figura 7. Trigger del Circuit Breaker a seguito di una chiamata ad un endpoint inesistente.

Evoluzione della Logica di Business

Gestione delle Soglie

L'API POST /interests è stata significativamente arricchita per supportare una gestione granulare delle preferenze utente. Oltre al codice aeroporto, l'utente può specificare le soglie high_value e low_value, con un'ulteriore controllo sulla correttezza di queste.

```

POST http://127.0.0.1/interests
Show Request

HTTP/1.1 400 BAD REQUEST
> (Headers) ...Content-Type: application/json...

{
  "Errore": "Richiesta Errata (400)",
  "details": "Errore logico: high_value deve essere maggiore di low_value"
}

Response file saved.
> 2025-12-20T122913.400.json

Response code: 400 (BAD REQUEST); Time: 8ms (8 ms); Content length: 108 bytes (108 B)

```

Figura 8. Controllo logico delle soglie.

Il sistema implementa ora una logica di **Upsert(Update or Insert)**:

- **Inserimento:** Se l'interesse è nuovo, viene creato con le soglie indicate.
- **Aggiornamento:** Se l'utente monitorava già quell'aeroporto, le nuove soglie sovrascrivono le precedenti. Questo permette agli utenti di raffinare i propri alert nel tempo senza dover cancellare e ricreare l'interesse.

```

POST http://127.0.0.1/interests
Show Request

HTTP/1.1 200 OK
> (Headers) ...Content-Type: application/json...

{
  "message": "Soglie aggiornate per l'aeroporto d'interesse KJFK per l'utente mario.rossi@unict.it"
}

Response file saved.
> 2025-12-20T122934.200.json

Response code: 200 (OK); Time: 29ms (29 ms); Content length: 99 bytes (99 B)

```

Figura 9. Aggiornamento delle soglie esistenti per un'utente. Inoltre, è stata introdotta la funzionalità

di **Visualizzazione** (GET /interests), che supporta due modalità:

- **Globale:** Restituisce la lista di tutti gli interessi nel sistema (utile per debug o amministrazione).

```
HTTP/1.1 200 OK
> (Headers) ...Content-Type: application/json...

[
  {
    "airport_code": "KJFK",
    "email": "mario.rossi@unict.it",
    "high_value": 100.0,
    "low_value": 5.0
  },
  {
    "airport_code": "EGLL",
    "email": "giovanni.contarino.gc@gmail.com",
    "high_value": 200.0,
    "low_value": 10.0
  }
]
```

Figura 10. JSON di tutti gli interessi presenti attualmente nel sistema.

- **Per Utente:** Filtrando tramite query parameter (?email=...), restituisce solo le configurazioni e le soglie attive per uno specifico utente.

```
HTTP/1.1 200 OK
> (Headers) ...Content-Type: application/json...

[
  {
    "airport_code": "KJFK",
    "email": "mario.rossi@unict.it",
    "high_value": 100.0,
    "low_value": 5.0
  }
]

Response file saved.
> 2025-12-20T123025.200.json

Response code: 200 (OK); Time: 9ms (9 ms); Content length: 92 bytes (92 B)
```

Figura 11. JSON degli interessi di uno specifico utente.

Consistenza e Pulizia

È stato mantenuto l'uso di **gRPC** per la comunicazione interna sincrona, estendendone l'uso per la manutenzione della consistenza dei dati. Quando un utente viene cancellato tramite DELETE /users, lo User Manager invoca via gRPC una procedura di **Cleanup** sul Data Collector, che provvede a rimuovere tutti gli interessi orfani associati a quell'email, garantendo la pulizia del database.

```
DELETE http://127.0.0.1/interests
Show Request

HTTP/1.1 200 OK
(Headers) ...Content-Type: application/json...

{
  "message": "Interesse per KJFK rimosso correttamente"
}

Response file saved.
> 2025-12-20T123053.200.json

Response code: 200 (OK); Time: 31ms (31 ms); Content length: 55 bytes (55 B)
```

Figura 12. Rimozione di un interesse per uno specifico utente.

```
GET http://127.0.0.1/interests?email=mario.rossi%40unict.it
Show Request

HTTP/1.1 200 OK
(Headers) ...Content-Type: application/json...

[]

Response file saved.
> 2025-12-20T123112.200.json

Response code: 200 (OK); Time: 9ms (9 ms); Content length: 3 bytes (3 B)
```

Figura 13. L'interesse non è più presente nel database.

